

# UA1 Projet1 G4 Les\_Hadoop\_Riders

---

## Table des matières

---

### Projet 1 — Déploiement et Exploitation d'un Cluster Hadoop avec Docker

Membres du groupe

Introduction

#### Partie 1 — Contexte théorique et architecture (5 pts)

1. Cas d'usage Big Data (1.5 pt)
2. HDFS vs système de fichiers classique (1.5 pt)
3. Rôle de YARN — ResourceManager et NodeManager (2 pts)

#### Partie 2 — Déploiement et manipulation pratique (15 pts)

- A. Mise en place de l'environnement Docker (4 pts)
- B. Manipulation avancée sur HDFS (6 pts)
- C. Exécution d'un job YARN (5 pts)

#### Partie 3 — Administration, monitoring et analyse (10 pts)

1. Interfaces web (4 pts)
2. Facteur de réplication (3 pts)
3. Retour d'expérience / troubleshooting (3 pts)

Conclusion

Annexes

- A. Glossaire des sigles
- B. Structure du code source
- C. Liens du projet

# Projet 1 — Déploiement et Exploitation d'un Cluster Hadoop avec Docker

|                |                                                                                                                           |
|----------------|---------------------------------------------------------------------------------------------------------------------------|
| Cours          | Bases de données massives avancées (IFM30522)                                                                             |
| Unité          | UA1 — Projet 1                                                                                                            |
| Groupe         | G4 — Les_Hadoop_Riders                                                                                                    |
| Date de remise | 4 octobre 2026                                                                                                            |
| Présentation   | 8 octobre 2026                                                                                                            |
| Image Docker   | leshadoopriders/hadoop-tp-g4:1.0                                                                                          |
| Docker Hub     | <a href="https://hub.docker.com/r/leshadoopriders/hadoop-tp-g4">https://hub.docker.com/r/leshadoopriders/hadoop-tp-g4</a> |
| Code source    | <a href="https://github.com/asinyopetro/Les_Hadoop_Riders_G4">https://github.com/asinyopetro/Les_Hadoop_Riders_G4</a>     |

## Membres du groupe

| Étudiant               | Étudiant                 |
|------------------------|--------------------------|
| Komla Petro Asinyo     | Kassoum Dene             |
| Joel Kazoni Tugirimana | Forbes Magène            |
| Frank A Simo Ngounou   | Wren Surprenant-Nicolson |

**Objectif du livrable :** déployer un cluster Hadoop multi-nœuds avec Docker, manipuler HDFS, exécuter un job YARN, publier l'image sur Docker Hub, et documenter les résultats avec captures d'écran (terminal et interfaces web).

## Introduction

Ce rapport présente le travail du groupe **Les\_Hadoop\_Riders (G4)** dans le cadre du Projet 1 de l'UA1. Nous avons choisi de déployer et d'exploiter un **cluster Hadoop** (HDFS + YARN) entièrement **conteneurisé avec Docker**.

La démarche suivie est la suivante :

1. Comprendre le besoin Big Data (cas e-commerce et 3V) et les rôles de HDFS / YARN.
2. Construire une image Docker basée sur **Hadoop 3.3.6** et lancer **1 master + 5 workers**.
3. Vérifier le cluster (5 DataNodes Live), publier l'image sur Docker Hub.
4. Manipuler HDFS (droits, blocs, lecture, fusion, corbeille, réplication).
5. Exécuter un job MapReduce (calcul de  $\pi$ ) et analyser les interfaces de monitoring.
6. Documenter les problèmes rencontrés et les solutions.

Le présent document suit la structure de l'énoncé (Parties 1, 2 et 3), avec les commandes exactes et les preuves (captures).

## Partie 1 — Contexte théorique et architecture (5 pts)

### 1. Cas d'usage Big Data (1.5 pt)

Nous avons choisi le secteur de l'**e-commerce** (commerce électronique : vente de produits ou services via Internet).

**Scénario** : une chaîne de magasins avec un site web doit conserver et analyser :

- les **transactions** (tickets de caisse, montants, magasins) ;
- les **logs** de navigation et d'erreurs du site ;
- les **mouvements de stock** ;
- éventuellement des **images produits**.

Chaque jour le volume grossit, les données arrivent souvent, et les formats changent. Une base relationnelle seule peut devenir insuffisante pour tout stocker et traiter à grande échelle.

### Les 3V du Big Data

Les **3V** sont les trois caractéristiques classiques utilisées pour décrire le Big Data :

| V               | Signification                     | Application e-commerce                                                         |
|-----------------|-----------------------------------|--------------------------------------------------------------------------------|
| <b>Volume</b>   | Quantité de données               | Millions d'événements, historique sur plusieurs années                         |
| <b>Vélocité</b> | Vitesse d'arrivée / de traitement | Commandes et paiements en continu ; réaction rapide (fraude, rupture de stock) |
| <b>Variété</b>  | Diversité des formats             | Tables CSV, JSON, logs texte, images produits                                  |

Dans ce contexte :

- **HDFS** (*Hadoop Distributed File System*) sert à **stocker** les données de façon distribuée sur plusieurs machines.

- **YARN** (*Yet Another Resource Negotiator*) sert à **lancer des traitements** (MapReduce, etc.) sur le cluster en allouant CPU et mémoire.

## 2. HDFS vs système de fichiers classique (1.5 pt)

Un **système de fichiers classique** (ex. NTFS sous Windows, ext4 sous Linux) stocke un fichier sur **une** machine. Les métadonnées et le contenu sont gérés par l'**OS** local. Si le disque tombe en panne, les données peuvent être perdues (sauf sauvegarde externe).

**HDFS**, au contraire, est conçu pour un **cluster** :

| Critère              | FS classique (NTFS, ext4...) | HDFS                                                |
|----------------------|------------------------------|-----------------------------------------------------|
| Emplacement          | Une seule machine            | Plusieurs machines (cluster)                        |
| Organisation         | Fichier entier sur un disque | Fichier découpé en <b>blocs</b> (souvent 128 Mo)    |
| Métadonnées          | Gérées par l'OS local        | Gérées par le <b>NameNode</b>                       |
| Contenu              | Sur le même disque           | Sur les <b>DataNodes</b>                            |
| Tolérance aux pannes | Faible (disque unique)       | Forte grâce à la <b>réplication</b> (ex. facteur 3) |

Le **NameNode** conserve la *namespace* (noms de fichiers, dossiers, emplacement des blocs). Les **DataNodes** stockent les blocs réels. Plusieurs copies d'un même bloc peuvent exister sur des nœuds différents : si un DataNode tombe, les données restent disponibles ailleurs.

**En pratique** : le FS local est centralisé ; HDFS est distribué et plus tolérant aux pannes grâce à la réplication.

## 3. Rôle de YARN — ResourceManager et NodeManager (2 pts)

**YARN** gère les **ressources** du cluster (CPU, mémoire) lorsqu'une application est soumise. Le flux typique est le suivant :

1. Un **client** soumet un job (ex. `yarn jar ... pi`).
2. Le **ResourceManager (RM)**, sur le master, reçoit la demande, connaît l'état des ressources, alloue des **conteneurs** et démarre l'**ApplicationMaster**.
3. Les **NodeManagers (NM)**, sur chaque worker, démarrent et surveillent les conteneurs **localement**, puis remontent l'état au RM.

4. L'**ApplicationMaster** pilote le job (demande de conteneurs pour les tâches Map/Reduce, suivi de l'avancement).

| Composant                     | Où ?             | Rôle                                               |
|-------------------------------|------------------|----------------------------------------------------|
| <b>ResourceManager (RM)</b>   | Master           | Orchestre et alloue les ressources                 |
| <b>NodeManager (NM)</b>       | Chaque worker    | Exécute et surveille les conteneurs sur sa machine |
| <b>ApplicationMaster (AM)</b> | Conteneur alloué | « Chef » du job en cours                           |

À retenir : le ResourceManager **orchestre** ; le NodeManager **exécute** localement.

## Partie 2 — Déploiement et manipulation pratique (15 pts)

### A. Mise en place de l'environnement Docker (4 pts)

#### Choix techniques

| Élément         | Choix du groupe                                    |
|-----------------|----------------------------------------------------|
| Version Hadoop  | <b>3.3.6</b>                                       |
| Image de base   | apache/hadoop:3.3.6                                |
| Image du groupe | leshadoopridders/hadoop-tp-g4:1.0                  |
| Orchestration   | Docker Compose ( <code>docker-compose.yml</code> ) |
| Architecture    | 1 master + <b>5</b> workers                        |
| Réseau          | hadoop-net (bridge)                                |

Chaque conteneur démarre via `entrypoint.sh` selon la variable `HADOOP_ROLE` ( `master` ou `worker` ). Les fichiers de configuration ( `core-site.xml` , `hdfs-site.xml` , `yarn-site.xml` , `mapred-site.xml` , `workers` ) sont copiés dans l'image.

## 1. Déploiement NameNode + 5 DataNodes (1.5 pt)

Nous avons déployé **six conteneurs** :

| Conteneur                         | Rôles                                        |
|-----------------------------------|----------------------------------------------|
| hadoop-master                     | NameNode, ResourceManager, SecondaryNameNode |
| hadoop-worker1 ... hadoop-worker5 | DataNode + NodeManager                       |

**Ports exposés sur la machine hôte :**

| Port | Service                          |
|------|----------------------------------|
| 9870 | Interface web HDFS (NameNode UI) |
| 8088 | Interface web YARN               |
| 9000 | HDFS RPC                         |

**Commandes de démarrage :**

```
docker compose up --build -d
docker ps
```

```
Windows PowerShell — Les_Hadoop_Riders_G4

PS C:\Users\asiny\Desktop\Les_Hadoop_Riders_G4> docker ps
NAME                    STATUS    PORTS
hadoop-worker2          Up 10 hours   8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker3          Up 10 hours   8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker5          Up 10 hours   8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker1          Up 10 hours   8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker4          Up 10 hours   8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-master           Up 10 hours   0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp

PS C:\Users\asiny\Desktop\Les_Hadoop_Riders_G4> docker exec -it hadoop-master hdfs dfsadmin -report
Configured Capacity: 5405585884160 (4.92 TB)
Present Capacity: 5032864239616 (4.58 TB)
DFS Remaining: 5032862023680 (4.58 TB)
DFS Used: 2215936 (2.11 MB)
DFS Used%: 0.00%
Replicated Blocks:
  Under replicated blocks: 0
  Blocks with corrupt replicas: 0
  Missing blocks: 0
  Missing blocks (with replication factor 1): 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
Erasure Coded Block Groups:
  Low redundancy block groups: 0
  Block groups with corrupt internal blocks: 0
  Missing block groups: 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0

-----
Live datanodes (5):

Name: 172.23.0.3:9866 (hadoop-worker5.les_hadoop_riders_g4_hadoop-net)
Hostname: hadoop-worker5
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
```

Figure 1 — Terminal : docker ps / dfsadmin

## 2. Vérification des DataNodes (1.5 pt)

```
docker exec -it hadoop-master hdfs dfsadmin -report
```

**Résultat observé : Live datanodes (5) — le cluster est opérationnel.**

Interface NameNode : <http://localhost:9870>

Cette commande confirme que les cinq DataNodes se sont bien enregistrés auprès du NameNode. Sans cela, le stockage HDFS ne serait pas réellement distribué.

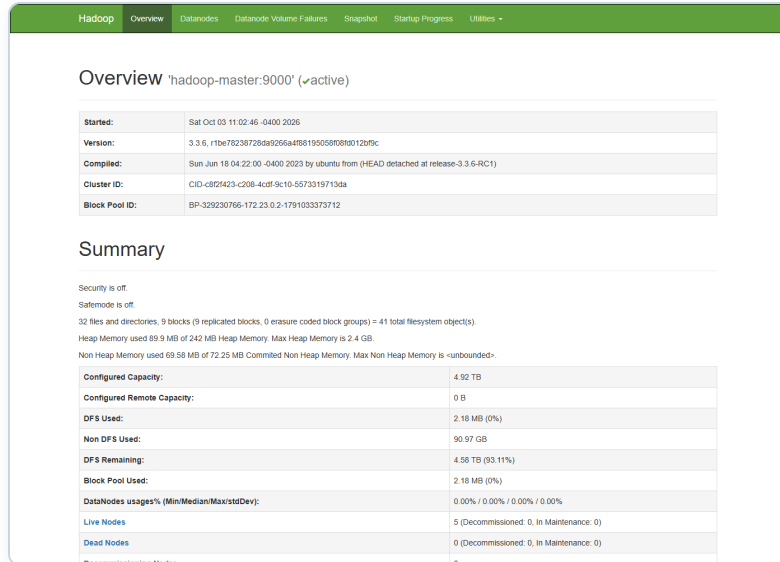


Figure 2 — UI NameNode (vue d'ensemble)

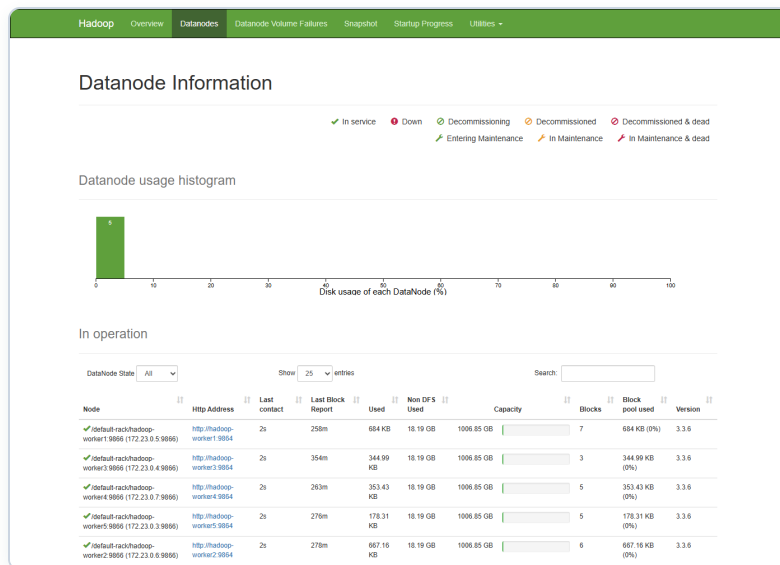


Figure 3 — Liste des DataNodes (5 Live)

### 3. Publication de l'image (1 pt)

```
docker login
docker push leshadoopridders/hadoop-tp-g4:1.0
```

**URL publique** : <https://hub.docker.com/r/leshadoopridders/hadoop-tp-g4>

**Note** : Docker Hub impose les **minuscules** dans le nom du dépôt, d'où **g4** plutôt que **G4** .

Toute personne disposant de Docker peut ensuite récupérer l'image et relancer le cluster à partir de notre dépôt GitHub / Compose.

## B. Manipulation avancée sur HDFS (6 pts)

Nous simulons des données de ventes e-commerce. Le fichier local `data/transactions.csv` contient 10 lignes (colonnes : ID, Date, Montant, Magasin).

### 4. Répertoires et droits (2 pts)

```
hdfs dfs -mkdir -p /data/ventes/2026
hdfs dfs -chmod 755 /data/ventes/2026
hdfs dfs -put -f /data/transactions.csv /data/ventes/2026/
hdfs dfs -chmod 644 /data/ventes/2026/transactions.csv
```

- `mkdir -p` crée l'arborescence.
- `chmod 755` sur le dossier autorise lecture/exécution pour les autres, écriture pour le propriétaire.
- `put` envoie le CSV vers HDFS.
- `chmod 644` sur le fichier le rend lisible sans le rendre exécutable.

### 5. Transfert et vérification des blocs (2 pts)

```
hdfs fsck /data/ventes/2026/transactions.csv -files -blocks -locations
hdfs dfs -stat "%n | taille=%b | replication=%r | block_size=%o"
/data/ventes/2026/transactions.csv
```

| Indicateur           | Valeur observée                            |
|----------------------|--------------------------------------------|
| Taille               | 377 octets                                 |
| Réplication initiale | 3                                          |
| Nombre de blocs      | 1 (fichier petit ; bloc par défaut 128 Mo) |
| État FSCK            | <b>HEALTHY</b>                             |

Un fichier très petit occupe quand même **un bloc** HDFS : c'est normal. La réplication à 3 signifie que ce bloc existe en trois copies sur le cluster.

```
root@hadoop-master:~# hdfs dfs -ls /data/ventes/2026
Found 2 items
-rw-r--r-- 2 hadoop supergroup 377 2026-10-03 13:30 /data/ventes/2026/transactions.csv
-rw-r--r-- 3 hadoop supergroup 567 2026-10-03 13:31 /data/ventes/2026/transactions_merged.csv
root@hadoop-master:~# hdfs dfs -cat /data/ventes/2026/transactions.csv | head -n 5
ID,Date,Montant,Agence
T001,2026-01-05,49.99,Montreal-Centre
T002,2026-01-06,129.50,Laval
T003,2026-01-07,15.25,Quebec
T004,2026-01-08,250.00,Montreal-Ouest
root@hadoop-master:~# hdfs fsck /data/ventes/2026/transactions.csv -files -blocks -locations 2x1 | head -n 25
Connecting to namenode via http://hadoop-master:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fdata%2Fventes%2F2026%2Ftransactions.csv
FSCK started by root (auth:SIMPLE) from /172.23.0.2 for path /data/ventes/2026/transactions.csv at Sun Oct 04 01:02:57 GMT 2026

/data/ventes/2026/transactions.csv 377 bytes, replicated: replication=2, 1 block(s): OK
0. BP-329230766-172.23.0.2-1791033373712:blk_1073741825_1001 len=377 Live_repl=2 [DataNodeInfoWithStorage[172.23.0.4:9866,DS-6b0bc5c8-0343-4441-977a-45186fdef03,DISK], DataNodeInfoWithStorage[172.23.0.6:9866,DS-f042a204-27cf-4970-b237-39c216ef47fb,DISK]]

Status: HEALTHY
Number of data-nodes: 5
Number of racks: 1
Total dirs: 0
Total symlinks: 0

Replicated Blocks:
Total size: 377 B
Total files: 1
Total blocks (validated): 1 (avg. block size 377 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 3
Average block replication: 2.0
Missing blocks: 0
Corrupt blocks: 0
```

Figure 4 — Terminal : commandes HDFS (fsck, stat, etc.)

## 6. Lecture et concaténation (1 pt)

```
hdfs dfs -cat /data/ventes/2026/transactions.csv | head -n 5
hdfs dfs -put -f /data/transactions_part2.csv /data/ventes/2026/
hdfs dfs -getmerge /data/ventes/2026 /tmp/transactions_merged.csv
hdfs dfs -put -f /tmp/transactions_merged.csv
/data/ventes/2026/transactions_merged.csv
```

Les 5 premières lignes sont lues directement depuis HDFS ( `cat` + `head` ). Ensuite, un second fichier est ajouté et les fichiers du dossier sont fusionnés avec `getmerge` , puis le résultat est réinjecté dans HDFS.

## 7. Suppression / corbeille (1 pt)

```
hdfs dfs -rm /data/ventes/2026/transactions_part2.csv
hdfs dfs -D fs.trash.interval=10080 -rm /data/ventes/2026/to_delete.csv
hdfs dfs -ls -R /user/hadoop/.Trash
# suppression définitive :
# hdfs dfs -rm -skipTrash /chemin/fichier
```

Sans `fs.trash.interval` , la suppression peut être **définitive**. Avec un intervalle > 0, le fichier passe dans `.Trash` (corbeille HDFS). L'option `-skipTrash` force la suppression définitive, sans passer par la corbeille.

## C. Exécution d'un job YARN (5 pts)

Pour valider YARN, nous avons lancé l'exemple officiel MapReduce de calcul de  $\pi$  (méthode de **Monte Carlo** : estimation à partir de points aléatoires).

```
yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar  
pi 4 1000
```

Les paramètres `4` et `1000` correspondent à **4 tâches map** et **1000 échantillons** par map.

| Champ       | Résultat                       |
|-------------|--------------------------------|
| Application | application_1791039798429_0001 |
| Nom         | QuasiMonteCarlo                |
| État final  | <b>SUCCEEDED</b>               |
| Estimation  | $\pi \approx$ <b>3.14</b>      |
| Paramètres  | 4 maps, 1000 samples           |

Ce résultat montre que le ResourceManager a alloué des ressources et que les NodeManagers ont bien exécuté les tâches jusqu'à la fin.

```
root@hadoop-master: ~# yarn application -list -appStates ALL
docker : 2026-10-04 01:03:01,304 INFO client.DefaultHadoopFollowerProxyProvider: Connecting to ResourceManager at
hadoop-master/172.23.0.2:8032
Au caractère C:\Users\asiny\AppData\Local\Temp\ps-script-67868ea5-b239-4ddd-997c-850c8a40b275.ps1:107 : 9
+ $yarn = docker exec hadoop-master bash -c 'yarn application -list -ap ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (2026-10-04 01:03:01.172.23.0.2:8032:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

Total number of applications (application-types: [], states: [NEW, NEW_SAVING, SUBMITTED, ACCEPTED, RUNNING, FINISHED, FAILED, KILLED] and tags:
[]):1
Application-Id      Application-Name      Application-Type      User      Queue      State
Final-State      Progress      Tracking-URL
application_1791039798429_0001      QuasiMonteCarlo      MAPREDUCE      root      default      FINISHED
SUCCEEDED      100%      http://hadoop-worker1:19888/jobhistory/job/job_1791039798429_0001
root@hadoop-master:~# yarn application -status application_1791039798429_0001 2>&1 | head -n 30
2026-10-04 01:03:05,625 INFO client.DefaultHadoopFollowerProxyProvider: Connecting to ResourceManager at hadoop-master/172.23.0.2:8032
2026-10-04 01:03:06,463 INFO conf.Configuration: resource-types.xml not found
2026-10-04 01:03:06,464 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
Application Report :
  Application-Id : application_1791039798429_0001
  Application-Name : QuasiMonteCarlo
  Application-Type : MAPREDUCE
  User : root
  Queue : default
  Application Priority : 0
  Start-Time : 1791070566554
  Finish-Time : 1791070642461
  Progress : 100%
  State : FINISHED
  Final-State : SUCCEEDED
  Tracking-URL : http://hadoop-worker1:19888/jobhistory/job/job_1791039798429_0001
  RPC Port : 41245
  AM Host : hadoop-worker1
  Aggregate Resource Allocation : 210745 MB-seconds, 245 vcore-seconds
  Aggregate Resource Preempted : 0 MB-seconds, 0 vcore-seconds
  Log Aggregation Status : DISABLED
```

Figure 5 — Terminal : sortie du job  $\pi$

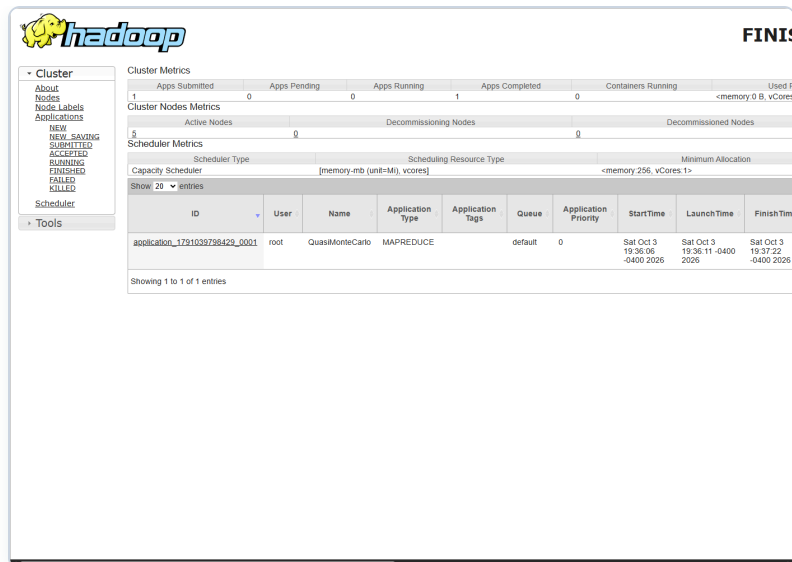


Figure 6 — UI YARN : liste des applications

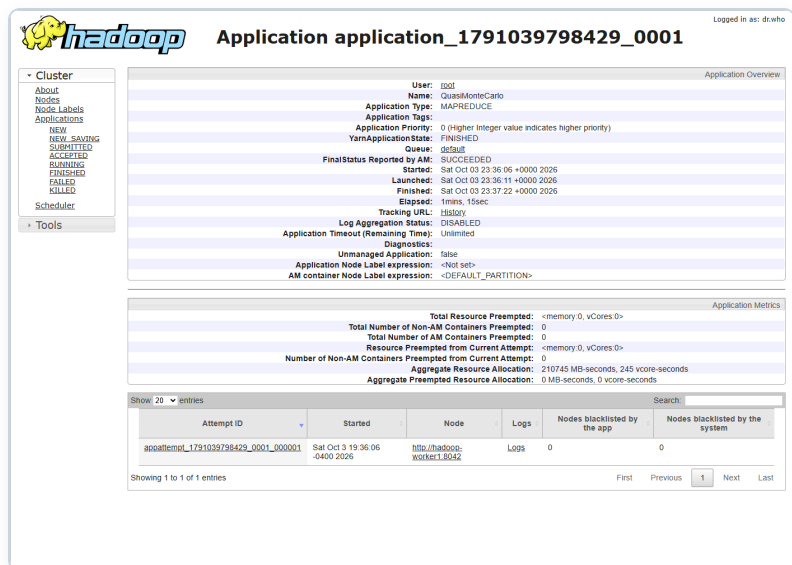


Figure 7 — Détail application FINISHED / SUCCEEDED

## Partie 3 — Administration, monitoring et analyse (10 pts)

### 1. Interfaces web (4 pts)

Les interfaces web permettent de contrôler le cluster sans rester uniquement en ligne de commande.

**NameNode** — <http://localhost:9870>

On observe **5 DataNodes** actifs. L'espace DFS utilisé reste faible (environnement de laboratoire, petits fichiers CSV). Aucun *missing block* (bloc manquant) n'a été observé pendant nos tests.

**YARN** — <http://localhost:8088>

L'application pi apparaît en état **FINISHED / SUCCEEDED**. Les métriques affichent l'allocation

mémoire et le temps d'exécution (environ **198589 MB-seconds** et **216 vcore-seconds** sur un des runs).

Ces UI sont complémentaires aux commandes `dfsadmin -report` et `yarn application -list`.

## 2. Facteur de réplication (3 pts)

```
hdfs dfs -setrep -w 2 /data/ventes/2026/transactions.csv
```

Après la commande, la réplication du fichier passe de **3 à 2**. Le NameNode met à jour la **cible** de réplication. Une copie de bloc devenue superflue est retirée sur les DataNodes. L'option `-w` (*wait*) attend que l'opération soit terminée avant de rendre la main.

Baisser la réplication économise de l'espace disque, mais réduit la redondance. L'augmenter fait l'inverse : plus de tolérance aux pannes, plus d'espace consommé.

## 3. Retour d'expérience / troubleshooting (3 pts)

| # | Problème               | Symptôme                                       | Cause                                              | Solution                                                                                   |
|---|------------------------|------------------------------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1 | Pull avant publication | <code>pull access denied</code>                | Compose téléchargeait une image pas encore sur Hub | <code>pull_policy: never</code> + build local, puis <code>docker push</code>               |
| 2 | Majuscules dans le tag | <code>repository name must be lowercase</code> | Docker Hub refuse G4                               | Tag final <code>leshadoopridders/hadoop-tp-g4:1.0</code>                                   |
| 3 | Build trop long        | Téléchargement bloqué                          | archive.apache.org très lent                       | Image de base <code>apache/hadoop:3.3.6</code> + notre config / <code>entrypoint.sh</code> |

Ces incidents ont permis de mieux comprendre le cycle de vie d'une image Docker (build → run local → push Hub → pull éventuel) et les contraintes de nommage des registres.

## Conclusion

Le groupe **Les\_Hadoop\_Riders** a livré un cluster Hadoop opérationnel en Docker :

- **Architecture** conforme : 1 master + 5 workers (DataNodes / NodeManagers).

- **HDFS** manipulé : arborescence, droits, fsck, lecture, getmerge, Trash, setrep.
- **YARN** validé : job MapReduce  $\pi$  en état **SUCCEEDED** ( $\pi \approx 3.14$ ).
- **Image publiée** : [leshadoopridders/hadoop-tp-g4:1.0](https://hub.docker.com/r/leshadoopridders/hadoop-tp-g4:1.0) sur Docker Hub.
- **Preuves** : captures terminal + UI NameNode / YARN dans ce rapport et dans le dépôt.

Ce travail montre que nous savons déployer, exploiter et administrer un environnement Hadoop conteneurisé, en reliant la théorie (3V, HDFS, YARN) à une mise en pratique documentée.

## Annexes

### A. Glossaire des sigles

| Sigle / terme     | Signification                                                         |
|-------------------|-----------------------------------------------------------------------|
| <b>HDFS</b>       | <i>Hadoop Distributed File System</i> — système de fichiers distribué |
| <b>YARN</b>       | <i>Yet Another Resource Negotiator</i> — gestionnaire de ressources   |
| <b>RM / NM</b>    | ResourceManager / NodeManager                                         |
| <b>3V</b>         | Volume, Vitesse, Variété                                              |
| <b>MapReduce</b>  | Modèle de calcul (phase Map puis Reduce)                              |
| <b>DFS</b>        | <i>Distributed File System</i>                                        |
| <b>UI</b>         | Interface utilisateur (web)                                           |
| <b>RPC</b>        | <i>Remote Procedure Call</i> (port 9000 HDFS)                         |
| <b>CSV</b>        | <i>Comma-Separated Values</i>                                         |
| <b>Trash</b>      | Corbeille HDFS                                                        |
| <b>setrep</b>     | Commande de changement du facteur de réplication                      |
| <b>fsck</b>       | Vérification de l'intégrité HDFS                                      |
| <b>Docker Hub</b> | Registre public d'images Docker                                       |

## B. Structure du code source

```
Les_Hadoop_Riders_G4/
├── Dockerfile
├── docker-compose.yml
├── entrypoint.sh
├── config/
│   ├── core-site.xml
│   ├── hdfs-site.xml
│   ├── yarn-site.xml
│   ├── mapred-site.xml
│   └── workers
├── data/
│   ├── transactions.csv
│   └── transactions_part2.csv
├── scripts/
│   ├── demo-hdfs.sh
│   └── demo-yarn-pi.sh
├── captures/
└── README.md
```

## C. Liens du projet

| Ressource           | URL                                                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Docker Hub          | <a href="https://hub.docker.com/r/leshadoopridders/hadoop-tp-g4">https://hub.docker.com/r/leshadoopridders/hadoop-tp-g4</a> |
| GitHub              | <a href="https://github.com/asinyopetro/Les_Hadoop_Riders_G4">https://github.com/asinyopetro/Les_Hadoop_Riders_G4</a>       |
| UI NameNode (local) | <a href="http://localhost:9870">http://localhost:9870</a>                                                                   |
| UI YARN (local)     | <a href="http://localhost:8088">http://localhost:8088</a>                                                                   |